

crp_qaoa.py — Complete Manual

A QAOA pipeline for the restricted Container Relocation Problem

Everything you need to install it, run it, read its output, understand how it works, and prove it is working correctly.

1. Quick start

macOS

macOS ships with a system Python that you should **not** install packages into. Use Homebrew's Python and a virtual environment.

Open Terminal and check what you have:

```
python3 --version
```

If that fails, or reports 3.8 or older, install a current Python:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"  
brew install python@3.12
```

Then set up a project folder and an isolated environment:

```
mkdir ~/crp && cd ~/crp  
# put crp_qaoa.py in this folder  
  
python3 -m venv .venv  
source .venv/bin/activate  
python -m pip install --upgrade pip  
python -m pip install numpy scipy
```

Your prompt now starts with `(.venv)`. Run the program:

```
python crp_qaoa.py --stacks "1,6,3,8; 5,2,7; 4; 9" --h 4 --show
```

Every new Terminal session needs `source ~/crp/.venv/bin/activate` again before running.

Apple Silicon note. On M1/M2/M3/M4 Macs, `numpy` and `scipy` install as native ARM wheels and are fast. No Rosetta needed. If you ever see “incompatible architecture”, you are mixing an Intel Python with ARM packages — delete `.venv` and rebuild it with the Homebrew Python.

Windows

```
cd "%HOME%\Desktop\Research\Python Code"  
py -3.12 -m venv .venv  
.\venv\Scripts\Activate.ps1  
python -m pip install numpy scipy  
python crp_qaoa.py --stacks "1,6,3,8; 5,2,7; 4; 9" --h 4 --show
```

The `ModuleNotFoundError` trap. VS Code may run a different interpreter than the `pip` on your PATH, so `pip install scipy` succeeds while the script still fails. Always install with `python -m pip`, using the *same* interpreter you run the script with. If VS Code shows the wrong one, press `Ctrl+Shift+P` → “Python: Select Interpreter” → choose the one inside `.venv`, then restart the terminal.

Optional: Qiskit

Only needed for `--backend aer`, `--noise`, and `--crosscheck`. Everything else runs on numpy and scipy alone.

```
python -m pip install qiskit qiskit-aer
```

If this fails to build, you are probably on Python 3.13, which may not have wheels yet. Build the environment with Python 3.12 instead.

2. What the program does

Given a bay of containers, it produces one row of results per instance, laid out like a benchmark results table.

```
python crp_qaoa.py --stacks "1,6,3,8; 5,2,7; 4; 9" --h 4 --show
```

```
lecture-bay (w=4, h=4, n=9)
t4 | 8 . . .
t3 | 3 7 . .
t2 | 6 2 . .
t1 | 1 5 4 9
   | - - - -
   S1 S2 S3 S4

Name      w  h  n Opt Pub QUBO QAOA Myop Solv Gap Lvl RIdx Lb Time Qb Bat Evals Popt Pfeas Sh95 RndF
SA% Val
lecture-bay 4 4 9 4 - 4 4 0 0 0 7 5 4 2.43 9 2 1635 0.1690 0.3423 32 0.1917
100 OK
```

That single row says: the true optimum is 4 relocations; the QAOA pipeline achieved 4; a levelling heuristic would have used 7; the widest quantum subproblem was 9 qubits; QAOA sampled the optimal plan 16.9% of the time; 32 shots give 95% confidence of seeing it; simulated annealing matched it at equal budget; and all validation checks passed.

3. The concepts behind it

This section explains *why* the program is built the way it is. Read it once; the rest of the manual assumes it.

3.1 The problem

Containers arrive at a port and are stacked in a **bay**: w vertical stacks, each holding at most h containers. A crane can only reach the **top** container of a stack — there is no way to pull one out of the middle.

Containers are labelled $1, 2, \dots, n$ by **retrieval priority**: container 1 leaves first, then 2, and so on. If container 1 is buried, every container above it must be lifted and put somewhere else first. Each such move is a **relocation**, and relocations are pure waste — crane time, fuel, berth time.

Restricted rCRP means only the container sitting on top of the *current target's* stack may be moved. No pre-emptive tidying elsewhere. This is how real terminals operate, because crane time is scarce.

The problem is NP-hard. With w stacks and roughly n relocations, the number of distinct plans grows like $(w-1)^n$ — for a 6-stack bay with 30 containers that is about 10^{21} plans, which at a billion per second would take roughly 30,000 years to enumerate.

3.2 Why not encode the whole problem

The standard exact formulations (Caserta et al., Galle et al.) index binary variables by *(container, container, time period)*, needing on the order of $n^2 T$ variables. The results file for `data3-3` reports **243 binary variables** for a 9-container bay, and **576** for a 12-container bay.

Converting 243 binaries into a QUBO and running QAOA on it would need 243 qubits plus slack variables. You cannot simulate that — a state vector would need 2^{243} complex numbers — and you could not verify it either.

So this program encodes **one batch at a time**: where does each blocker above the current target go? That is $(\text{number of blockers}) \times (\text{available destination stacks})$ variables, bounded by $(h-1)(w-1)$. For the same bay: **8 qubits instead of 243**.

After the batch is executed, the bay is re-examined and the next batch is encoded. This is a **rolling horizon**, exactly how real yard schedulers operate.

The price is **myopia**: the encoding cannot see that a cheap move now might cost more later. The program measures that price rather than hiding it (see §3.8).

3.3 QUBO — the required input format

QUBO stands for *Quadratic Unconstrained Binary Optimisation*. Choose $x_1, \dots, x_n \in \{0, 1\}$ to minimise

$$C(x) = \sum_i Q_{ii} x_i + \sum_{i < j} Q_{ij} x_i x_j + \text{const}$$

Three rules, and only three: variables are binary, terms are at most quadratic, and there are **no constraints** — every rule must live inside the cost function.

A useful identity: for binary variables $x^2 = x$, so squares collapse into linear terms. This is used constantly when expanding penalties.

3.4 The four cost terms

The program builds the cost by **counting physical events**, not by deriving algebra. Let $x_{ij} = 1$ mean “blocker i goes to stack j ”.

(1) Burial. If you place blocker i on a stack that already contains a container due out *sooner*, blocker i will have to be lifted off again. Let $w_{ij} = 1$ in that case. Cost: $\sum_{ij} w_{ij} x_{ij}$.

(2) Mis-stack. Blockers are moved in order, so the one moved earlier ends up lower. If two land on the same stack and the lower one has the smaller label, that is another future move. Let $v_{ik} = 1$ when blocker i (moved first) is due out before blocker k .

The double-count correction. Writing this term as $v_{ik} x_{ij} x_{kj}$ is wrong. If blocker k already buries a pre-existing smaller container on that stack ($w_{kj}=1$), then k must be relocated anyway — charging it again for burying blocker i counts the **same crane move twice**. The consequence is that the objective can *exceed* the true relocation count, so it is not a valid lower bound. Measured on 120 random instances, the uncorrected form is a valid lower bound in only **94%** of cases. The fix is one factor: $v_{ik}(1 - w_{kj}) x_{ij} x_{kj}$. With it, validity rises to **100%**. Use `--naive-pair` to reproduce the uncorrected version for comparison.

(3) One-hot. Nothing so far stops the optimiser from setting every variable to zero — cost zero, physically meaningless. Add $A(1 - \sum_j x_{ij})^2$ for each blocker. This is zero when exactly one variable is 1, and at least A otherwise.

(4) Capacity. At most `free[j]` blockers may land on stack j . Three cases:

- `free ≥ nB` — the rule can never bite. No penalty needed.
- `free == 1` — “at most one” is exactly a pairwise penalty $B \sum_{i < k} x_{ij} x_{kj}$.
- otherwise — needs **slack variables**. The program adds binaries y_t with weights that represent every integer in $[0, \text{free}]$ and nothing above, then penalises $B(\sum_i x_{ij} + \sum_t w_t y_t - \text{free})^2$. This costs 1–2 extra qubits per binding stack but is exact for any free-slot count.

`--capacity strict` disables slack variables and skips instances it cannot represent exactly. `--capacity slack` (the default) handles everything.

3.5 Choosing the penalty weights

The rule of thumb: A must exceed the largest saving a solution could get by breaking the constraint. Too small and the optimum is illegal. **Too large and the energy landscape becomes a needle in a haystack** — legal solutions get squashed into a thin band at the bottom while penalties dominate the spectrum, and QAOA performs worse.

The program defaults to $A = B = 2$. The professional workflow is to sweep several values, brute-force check which still give the correct unique ground state, and use the smallest that works. Use `--penalty-a` and `--penalty-b` to run that sweep.

3.6 The Ising form — and why it is optional

Physics speaks in spins $z \in \{-1, +1\}$ rather than bits. The substitution $x_i = (1 - z_i)/2$ turns the QUBO into an **Ising energy** $\sum_i h_i z_i + \sum_{i < j} J_{ij} z_i z_j + \text{offset}$, and replacing each z_i by the Pauli operator Z_i gives the quantum cost Hamiltonian.

Here is the simplification that removes the hardest part of the usual presentation:

The QUBO and Ising forms differ by an **additive constant**. In QAOA the cost layer multiplies amplitude z by $e^{-i\gamma E(z)}$, so shifting every energy by a constant multiplies the whole state by one fixed factor — a **global phase**. That changes no probability, and it commutes with everything afterwards.

Therefore running QAOA on the QUBO cost gives an **identical** measurement distribution to running it on the Ising Hamiltonian. The Ising rewrite is required only to build a *gate circuit* for real hardware.

The program provides `qubo_to_ising()` and uses it for the Aer backend and for validation check V3, but the default numpy backend works directly with the QUBO cost.

The cost Hamiltonian itself needs no operator theory to define:

$\{C\}$ is the $2^n \times 2^n$ diagonal matrix whose entry in row z , column z is the cost $E(z)$.

“The optimal solution is the ground state of $\{C\}$ ” then means nothing more than “the cheapest plan is the smallest number on the diagonal.”

3.7 QAOA

Pick a depth p and $2p$ angles. The QAOA state is

$$|\psi_p\rangle = e^{\wedge\{-i\beta_p\}\{B\}} e^{\wedge\{-i\gamma_p\}\{C\}} \cdots e^{\wedge\{-i\beta_1\}\{B\}} e^{\wedge\{-i\gamma_1\}\{C\}} |+\rangle^{\otimes n}$$

Three ingredients, each elementary:

1. **Start** — all 2^n amplitudes equal to $2^{-n/2}$: every candidate plan at once, none preferred.

2. **Cost layer** — $\{C\}$ is diagonal, so this just multiplies amplitude z by $e^{-iy E(z)}$. One line of code. It stamps a phase on each plan according to its cost, but moves no amplitude.
3. **Mixer layer** — $\{B\} = \sum_i X_i$, and the X_i act on different qubits, so this factorises into a 2×2 rotation applied to each pair of amplitudes differing in one qubit.

The division of labour: the cost layer marks good solutions with phases; the mixer moves amplitude between solutions so those phases can interfere. Cost without mixer does nothing visible; mixer without cost does nothing useful. Alternate them and good solutions accumulate amplitude.

A classical optimiser (COBYLA) then tunes the angles. The landscape is non-convex, so a single run from a random start is a coin flip — hence `--restarts`, which is standard practice in the QAOA literature.

What QAOA actually gives you. Not an oracle — a **biased random sampler**. You take a few thousand cheap shots, decode each into a plan, discard illegal ones, evaluate the rest classically in microseconds, and keep the best. Any bitstring that decodes to a legal plan is a legal plan; the quantum part cannot fool you. That is why a success probability of a few percent is already operationally useful.

3.8 CVaR

Minimising the *mean* energy is a slightly odd objective, because you do not care about the average sampled plan — you care about the **best** one. CVaR (Conditional Value at Risk) replaces the mean with the average energy of only the cheapest α fraction of the distribution.

The optimiser is then rewarded for growing a good *tail* rather than shifting the whole distribution. It typically doubles success probability, even though it makes the reported mean energy look worse. Default `--alpha 0.15`; use `--alpha 1.0` for standard mean-energy QAOA.

3.9 The myopia/solver decomposition — the key idea in this program

Suppose the pipeline uses 1.75 more relocations than optimal. That number is useless on its own, because it mixes two unrelated failures:

- the **encoding** might be myopic — the one-batch horizon cannot see far enough;
- the **solver** might be weak — QAOA failed to find its own QUBO’s optimum.

So the program runs the rolling horizon **twice**: once solving each batch *exactly* by brute force, and once solving it with QAOA.

Quantity	Formula	Blames
Myop	<code>QUBO - Opt</code>	the encoding
Solv	<code>QAOA - QUBO</code>	the solver
Gap	<code>QAOA - Opt = Myop + Solv</code>	both

This was verified behaviourally: raising `--restarts` from 4 to 16 shrank `Solv` from +1.25 to +1.00 and left `Myop` unchanged at +0.50 — which is precisely what each is supposed to measure.

3.10 Baselines — choosing them honestly

Comparing success probability against uniform guessing over all 2^n bitstrings flatters the result, because most bitstrings do not even decode to a legal plan. The stricter and fairer baseline is uniform guessing over **feasible plans only**.

The difference is not cosmetic. On the same instances the apparent advantage was $\sim 3\times$ against the weak baseline and **0.42x** — i.e. *worse than guessing* — against the strict one. The program reports the strict baseline as `RndF`.

It also runs **simulated annealing at matched evaluation budget** (`SA%`). Both methods get the same number of cost-function evaluations; who finds the optimum more often? This column exists so that a quantum-speedup claim cannot be made by accident.

4. Tour of the code

The file is one module in twelve numbered sections.

§	Contents	Key functions
1	Instances — the <code>Instance</code> dataclass, benchmark file parser, random generator, ASCII renderer	<code>parse_instance_file</code> , <code>random_instance</code>
2	Exact solver — memoised exhaustive search giving ground truth	<code>exact_relocations</code> , <code>blocking_lower_bound</code>
3	Heuristic baselines — levelling and reshuffle-index	<code>heuristic_relocations</code>
4	Batch extraction — identify the target, its blockers, and candidate destinations	<code>extract_batch</code>
5	QUBO — the four cost terms, slack variables, exhaustive energy evaluation, decoding	<code>build_qubo</code> , <code>all_energies</code> , <code>decode</code>
6	Ising conversion — the $x=(1-z)/2$ substitution	<code>qubo_to_ising</code> , <code>ising_energies</code>
7	QAOA — numpy state vector, Qiskit circuit builder, Aer backend, CVaR, optimiser loop	<code>qaoa_probs_numpy</code> , <code>AerBackend</code> , <code>run_qaoa</code>
8	Metrics and baselines — success probability, shot counts, simulated annealing	<code>metrics</code> , <code>simulated_annealing</code>
9	Validation — the V1-V5 suite and the falsification control	<code>validate_batch</code> , <code>falsification_control</code>
10	Pipeline — the rolling-horizon driver under both policies	<code>_rolling</code> , <code>solve_instance</code>
11	Results table — fixed-width and CSV output	<code>format_table</code> , <code>write_csv</code>
12	Command line	<code>main</code>

The two functions worth reading closely

`exact_relocations` (§2) is your ground truth. It recursively tries every legal relocation of the blocking container and takes the minimum, memoising on bay state. Because it only ever moves the top of the target's stack, it implements the *restricted* variant by construction.

`qaoa_probs_numpy` (§7) is the entire quantum algorithm in thirteen lines. Note what is *absent*: no Pauli algebra, no CNOT-RZ-CNOT identity, no factor-of-2 convention, no endianness handling. Those exist only to compile QAOA into gates. Simulating the defining equation needs none of them.

5. Running it

Instance sources — choose exactly one

```
# a bay typed in directly (stacks bottom -> top, ';' between stacks)
python crp_qaoa.py --stacks "1,6,3,8; 5,2,7; 4; 9" --h 4 --name lecture-bay

# one benchmark instance file, with its published optimum
python crp_qaoa.py --file data3-3-11 --published 3

# a whole class of instances
python crp_qaoa.py --dir instances/group4 --glob "data3-3-*" --csv data33.csv

# random instances
python crp_qaoa.py --random 40 --w 3 --h 5 --n 9 --seed 1
```

All options

Option	Default	Meaning
<code>-p, --depth</code>	3	QAOA depth (number of layers)
<code>--alpha</code>	0.15	CVaR tail fraction; 1.0 = standard mean-energy QAOA
<code>--restarts</code>	16	random restarts per optimisation
<code>--maxiter</code>	400	COBYLA iterations per restart
<code>--shots</code>	4096	shots used to pick the final plan
<code>--penalty-a</code>	2.0	one-hot penalty weight A
<code>--penalty-b</code>	2.0	capacity penalty weight B
<code>--capacity</code>	slack	slack (general) or strict (skips some instances)
<code>--naive-pair</code>	off	reinstate the double-counting mis-stack term
<code>--max-qubits</code>	20	refuse batches wider than this
<code>--backend</code>	numpy	numpy or aer
<code>--aer-shots</code>	0	Aer shots per evaluation (0 = exact state vector)
<code>--noise</code>	0.0	depolarising error per 2-qubit gate (implies aer)
<code>--crosscheck</code>	off	report max $ P_{\text{aer}} - P_{\text{numpy}} $
<code>--seed</code>	0	master random seed
<code>--repeats</code>	1	independent seeds per instance
<code>--sweep-depth</code>	—	e.g. 1 2 3 4 5 6
<code>--no-verify</code>	off	skip the validation suite (faster)
<code>--no-sa</code>	off	skip the simulated-annealing baseline
<code>--no-exact</code>	off	skip the exact solver (for bays too large to solve)
<code>--csv</code>	—	write results to a CSV file
<code>--show</code>	off	draw each bay before running
<code>--quiet</code>	off	suppress tables, keep summaries

The main experiment

```
python crp_qaoa.py --dir instances/group4 --glob "data3-3-*" \
--sweep-depth 1 2 3 4 5 6 --repeats 10 --csv sweep.csv
```

<pre class="wide">
40 instances × 10 seeds × 6 depths. Add a second pass with `--alpha 1.0` to compare CVaR against mean-energy QAOA.

****Restarts versus repeats – do not confuse them.**** `--restarts` is part of the **algorithm**: COBYLA is a local method on a non-convex landscape, so you restart and keep the best. `--repeats` is your **sample size**: independent seeds giving the spread you report as error bars. Ten repeats across forty instances gives 400 data points per condition, which is ample; report median and interquartile range, never the mean of a single run.

6. Reading the output

```
<pre class="wide">Name          w   h   n Opt  Pub  QUB0  QAOA  Myop  Solv  Gap  Lvl  RIDx  Lb   Time  Qb  Bat  Evals  Pop
Pfeas Sh95  RndF  SA%  Val
lecture-bay  4  4  9  4  -  4  4  0  0  0  7  5  4  2.43  9  2  1635 0.1690 0.3423 32 0.
100 OK</pre>
```

Instance identity

```
Column	Meaning
`Name`	instance name or filename
`w`	number of stacks
`h`	tier limit (maximum stack height)
`n`	number of containers
```

Solution quality – the important block

```
Column	Meaning	How to read it
`Opt`	exact minimum relocations from the exhaustive solver	**Ground truth.** Everything else is judged against this
`Pub`	published optimum, if supplied with `--published`	Must equal `Opt`. If it does not, you have a bug – stop
`QUB0`	rolling horizon with each batch solved *exactly*	How good the **encoding** is, with solving perfect
`QAOA`	rolling horizon with each batch solved by QAOA	What the **full pipeline** achieves
`Myop`	`QUB0 – Opt`	Cost of the one-batch horizon. Blames the **encoding**. Does not improve with more restarts
`Solv`	`QAOA – QUB0`	QAOA missing its own QUB0's optimum. Blames the **solver**. Shrinks with more restarts
`Gap`	`QAOA – Opt` = `Myop + Solv`	Total. Never report this without the split
`Lvl`	levelling heuristic, full sequence	Classical baseline. A sensible rule that is blind to interactions
`RIDx`	reshuffle-index heuristic, full sequence	Stronger classical baseline
`Lb`	blocking lower bound	Every container above a smaller label must move at least once. `Opt ≥ Lb` always
```

Cost and scale

```
Column	Meaning	How to read it
`Time`	wall-clock seconds for the whole instance	Both rolling-horizon passes included
`Qb`	widest batch, in qubits	The binding constraint on runtime. See §8
`Bat`	number of batches QAOA solved	Batches where the target was already clear are free and not counted
`Evals`	total cost-function evaluations	The budget simulated annealing is matched against
```

Quantum performance

```
Column	Meaning	How to read it
`Popt`	mean probability of sampling the batch optimum	**The headline quantum number.** 0.1690 = 16.9% per shot
`Pfeas`	mean probability mass on legal plans	**Diagnoses your penalties.** Near zero means $A$ is too small, or the
search space is mostly rubbish		
`Sh95`	shots for 95% confidence of seeing the optimum	The operational reading: "32 shots suffice"
`RndF`	uniform over *feasible* plans	**The strict baseline.** Compare `Popt` against this, not against  $1/2^n$
`SA`	% of batches where simulated annealing found the optimum at matched budget	100% means classical wins outright. Rep
it		
`Val`	`OK` or `FAIL` for the V1–V5 suite	Any `FAIL` invalidates every number in that row
```

The summary block

```
</pre>
instances          1 total, 1 completed
QAOA == exact optimum 100%  mean total gap +0.000 relocations
... myopia (one-batch horizon) +0.000  [100% of instances unaffected]
... solver (QAOA vs its QUB0) +0.000  [100% of instances unaffected]
levelling          0% optimal, mean gap +3.000
reshuffle-index    0% optimal, mean gap +1.000
median P_opt 0.1690  median random-feasible baseline 0.1917  ratio 0.88x
simulated annealing at matched budget: optimum in 100% of batches
```

The lines that matter for a paper: the **myopia/solver split** (the encoding result versus the solver result), the **heuristic comparison** (is the encoding a better decision rule than standard practice?), and the **ratio against the random-feasible baseline** (is QAOA doing anything?).

A ratio below 1.0 means QAOA underperformed guessing among legal plans. That happens on very small instances where there are few legal plans to begin with, and it is worth reporting rather than hiding.

7. Setting up benchmark instances

The published results give you optima; the layouts come from the corresponding instance archive. Match them by filename.

File format

One text file per instance:

```
3 5 9      <- w h n (matching the w, h, n columns in the results file)
3 4 7 2    <- stack 1: 3 containers, listed bottom -> top
3 9 1 8    <- stack 2
3 5 3 6    <- stack 3
```

The parser auto-detects whether each stack line carries a leading count, so this also works:

```
3 5 9
4 7 2
9 1 8
5 3 6
```

Blank lines and `#` comments are ignored.

Which classes to use

For the `group 4` set, $n = w \times (h-2)$, and the worst-case qubits per batch is $(h-1)(w-1)$:

Class	w	h	n	Instances	Max qubits
data3-3-*	3	5	9	40	8
data3-4-*	4	5	12	40	12
data3-5-*	5	5	15	40	16
data3-6-*	6	5	18	40	20 — too slow

The first three give an $8 \rightarrow 12 \rightarrow 16$ qubit ladder with 120 instances, all with proven optima.

The first thing to run

```
python crp_qaoa.py --dir instances/group4 --glob "data3-3-*" --no-sa --quiet --csv check.csv
```

Confirm the summary reports **100% agreement** between `opt` and the published values. If it does, that sentence belongs in your paper. If it does not, stop and find the bug before running anything else.

8. Checking that it works

8.1 The validation suite (the `val` column)

Run automatically on every batch unless you pass `--no-verify`.

Check	What it establishes
V1	Every infeasible assignment is penalised by at least A — so no illegal plan can masquerade as a good one
V2	The minimum-energy state decodes to a legal plan
V3	Ising energy equals QUBO cost on all 2^n states
V4	$\gamma = 0$ reproduces the uniform distribution exactly
V5	Probabilities sum to 1

V3 is the important one. It is not a spot check — it compares both expressions at *every* point of a finite domain. For a finite domain, exhaustive verification **is** a proof (proof by exhaustion). You are not approximating a derivation; you are doing something stronger than printing algebra, because algebra on paper does not establish that your *code* matches it.

V4 doubles as exposition. At $\gamma = 0$ the cost layer does nothing, so the algorithm must return uniform randomness. That one line simultaneously proves the cost layer is wired in and explains to a reader what the random-guessing baseline is.

8.2 The falsification control

A test that has never been shown to fail is a decoration, not a test.

```
import crp_qaoa as m
inst = m.Instance("t", 4, 4, 9, ((1,6,3,8),(5,2,7),(4,),(9,)))
b = m.extract_batch(inst.bay, inst.h)
q = m.build_qubo(inst.bay, b)
bits, E = m.all_energies(q)
print(m.falsification_control(q, bits, E)) # must print True
```

This corrupts one Ising field and confirms V3 then **fails**. Verified output: `True` . Run it once and report it.

8.3 The backend cross-check

```
python crp_qaoa.py --stacks "1,6,3,8; 5,2,7; 4; 9" --h 4 \
--backend aer --restarts 2 --maxiter 80 --crosscheck
```

Builds the same QAOA as a Qiskit gate circuit — with Pauli terms, CNOT-RZ-CNOT sandwiches, the factor-of-2 in `rz`, and the little-endian bit order all handled — and compares it against the direct state-vector method.

Measured result: **max $|\mathbf{P}_{\text{aer}} - \mathbf{P}_{\text{numpy}}| = 3.33 \times 10^{-16}$** , i.e. machine precision.

This is worth a sentence in your paper. Two independent implementations agreeing is far stronger evidence than one implementation agreeing with itself.

8.4 Expected values on the reference instance

```
--stacks "1,6,3,8; 5,2,7; 4; 9" --h 4
```

Quantity	Expected
Opt	4
Lb	4
Lvl	7
RIdx	5
Qb	9
Val	OK
Popt at p=3	roughly 0.10 – 0.25

`Opt`, `Lb`, `Lvl`, `RIdx` and `Qb` are deterministic and must match exactly. `Popt` varies between runs: QAOA is a randomised, non-convex optimisation, so your numbers will land *near* these rather than exactly on them. If `Popt` comes out near $1/2^9 = 0.002$, something is broken.

8.5 Behavioural checks

Two properties you can verify yourself, which confirm the decomposition measures what it claims:

- **Raise** `--restarts` from 4 to 16. `Solv` should shrink; `Myop` should not move at all. Verified: `Solv` went $+1.25 \rightarrow +1.00$, `Myop` stayed at $+0.50$.
- **Add** `--noise 0.01`. `Popt` should fall and `Sh95` should rise. Verified: `Popt` $0.169 \rightarrow 0.117$, `Sh95` $32 \rightarrow 322$.

9. Performance and sizing

Worst-case qubits per batch is $(h-1)(w-1)$:

w h	5	6	7
3	8	10	12
4	12	15	18
5	16	20	24
6	20	25	30

Measured, one batch, p=3, 6 restarts $\times$ 300 iterations, numpy backend:

Qubits	State vector	Time per batch
12	0.1 MB	1.5 s
16	1 MB	29 s
20	17 MB	6.4 min

Memory is never the constraint — time is. Twenty qubits is only 17 MB, but each optimisation needs hundreds of evaluations. Keep $(h-1)(w-1) 12$ for sweeps and reserve 16 qubits for a single showcase instance.

Backend cost, measured on 9 qubits at $p=3$:

Backend	Per evaluation	Relative	Full optimisation (6400 evals)
numpy	0.51 ms	1×	3.3 s
Aer state vector	4.35 ms	8.5×	27.8 s
Aer, 4096 shots	14.07 ms	27.6×	~90 s

Aer is not more accurate — it agrees to 3×10^{-16} . It is the same algorithm compiled to gates. **Use numpy for sweeps; use Aer for the cross-check and the noise study.**

10. Troubleshooting

Symptom	Cause and fix
ModuleNotFoundError: No module named 'scipy'	<code>pip</code> installed into a different interpreter. Use <code>python -m pip install numpy scipy</code> with the same interpreter you run the script with
labels must be exactly 1..n	The bay does not contain each label exactly once. Check for duplicates, gaps, or a wrong <code>n</code> in the file header
expected N stacks, got M	The header's <code>w</code> disagrees with the number of stack lines
stack (...) exceeds tier limit	A stack is taller than <code>h</code> . Check whether the benchmark's <code>h</code> means <i>filled tiers</i> or <i>tier limit</i> — for group 4, $n = w(h-2)$, so the bay is filled two tiers below the limit
batch needs N qubits > max_qubits	The bay is too wide. Raise <code>--max-qubits</code> if you can afford the time, or use a smaller class
out of scope (capacity=strict)	Expected with <code>--capacity strict</code> . Switch to <code>--capacity slack</code> (the default)
stuck: no destination	Every other stack is full. The instance is genuinely infeasible under the restricted rule
Very slow	Check the <code>Qb</code> column. Each extra qubit doubles the state vector. Also confirm you are not on <code>--backend aer</code>
Popt near $1/2^n$	QAOA is doing nothing. Check <code>Val</code> first; then check whether <code>--penalty-a</code> is far too large
Pfeas near zero	Penalties are misconfigured, or the batch has almost no legal plans
Qiskit will not install	Likely Python 3.13. Build the environment with Python 3.12

11. Limitations to state in any write-up

The one-batch formulation is myopic by construction. That is what `Myop` measures — it is a property of the encoding, not a bug. A full-sequence encoding would need hundreds of qubits.

Simulation is noiseless by default. No gate error, no decoherence, no connectivity limits, unless you pass `--noise`. Real hardware would be substantially worse.

Nothing here demonstrates a quantum speedup. The `SA%` column exists precisely so that claim cannot be made by accident. In testing, simulated annealing at matched budget found the optimum in essentially every batch.

Capacity handling is exact but costs qubits. Slack variables add 1-2 qubits per binding stack, which shows up in the `Qb` column.

Exhaustive verification caps instance size. `all_energies` builds all 2^n states, which is what makes V1-V3 proofs rather than spot checks — but it is also why batches above about 20 qubits are refused.